CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Method Note and Data Sources

Team 20

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Deependra Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

1. The method, and why this one

1.1 What EVM is for

A conventional monthly cost report compares two numbers: what was budgeted to have been spent by now, and what has actually been spent. The gap between them is reported as a variance. The trouble is that this single comparison conflates two entirely different problems.

A project that has spent ₹4.00 crore against a budgeted ₹4.02 crore looks healthy. It is healthy if it has also done ₹4.00 crore of work. It is in serious difficulty if it has done ₹3.47 crore of work, because it is then both behind programme and overspending, and the apparent agreement between budget and spend is a coincidence of two errors pointing in opposite directions. That is precisely the position of the case study project used throughout this work.

EVM resolves this by introducing a third measure. Instead of two numbers there are three, all in the same units:

Measure	Definition	The question it answers
PV Planned value	Budgeted cost of work **scheduled**	What should have been done by now?
EV Earned value	Budgeted cost of work **performed**	What has actually been done?
AC Actual cost	Cost incurred doing it	What did it cost?

Every other EVM quantity is a comparison of two of these three. EV against AC gives cost performance; EV against PV gives schedule performance. The whole method follows from having the third number.

1.2 Measures implemented

Variances. CV = EV − AC. SV = EV − PV. Negative is unfavourable throughout. This convention is worth stating because some contractor reporting formats invert it.

Indices. CPI = EV / AC, the rupees of budgeted work returned per rupee spent. SPI = EV / PV, the work done per unit of work scheduled. Below 1.00 is unfavourable.

Forecasts. Four standard estimate-at-completion methods, which differ in exactly one assumption — what is believed about the cost performance of the work that remains:

Method	Formula	Assumption
EAC1	BAC / CPI	Performance to date is typical and continues
EAC2	AC + (BAC − EV)	The variance was a one-off; the rest runs at budget
EAC3	AC + (BAC − EV) / (CPI × SPI)	Both cost and schedule pressure continue
EAC4	AC + (BAC − EV) / (0.8·CPI + 0.2·SPI)	Cost dominates, schedule partly weighted

From these: ETC = EAC − AC, VAC = BAC − EAC, and TCPI = (BAC − EV) / (EAC − AC), the efficiency the remaining work must achieve to hit a given target.

The dashboard presents all four together rather than choosing one. A single EAC implies a precision the method does not have. On the case study the spread between the most and least optimistic is ₹3.47 crore on a ₹14 crore project, and that spread *is* the honest answer: the method cannot narrow it without a judgement about whether the rock excavation overrun was a closed, one-off event.

1.3 Earned Schedule, and why we added it

Classical schedule variance has a defect that matters most on exactly the projects where schedule control matters most.

SV and SPI are measured in **currency**, not time. At completion, EV necessarily equals BAC and PV necessarily equals BAC. Therefore SV returns to zero and SPI returns to 1.00 — on every project, however late it finished. The indices improve through the final third of a project, and the improvement is an artefact of the arithmetic rather than a recovery. A planner relying on SPI alone receives his most reassuring reading at the moment the project is latest.

Earned Schedule (Lipke, 2003) asks a different question. Rather than "how much value has been earned compared with the value scheduled", it asks "**at what point on the baseline was this much value scheduled to have been earned**". That point is a time, so the resulting measures are in time:

```
ES      the baseline time at which cumulative PV equals current EV
AT      actual time elapsed at the data date
SV(t) = ES − AT        schedule variance, in periods
SPI(t) = ES / AT       time-based schedule efficiency
```

On the case study at Period 6, classical SPI reads **0.863** and SPI(t) reads **0.893**. More usefully, SV(t) is **−0.64 months**, which the dashboard states as "about 19 days behind programme". A site manager can act on nineteen days. Nobody can act on 0.863.

We implement **both** and display both, with the gap explained, rather than silently preferring one. The interpolation assumes planned value accrues linearly within a reporting period — a mild assumption over a monthly period on building work, and one that would not be safe on very long periods or very lumpy spend.

1.4 How earned value is measured

By the **percent-complete method** at work-package level:

```
EV(package) = cumulative percent complete × budget at completion of that package
```

This is the appropriate default for building work, where progress is measured as quantity executed against quantity in the bill. The alternatives — 0/100, 50/50 and milestone weighting — are better suited to short packages and to procurement, and would have forced an arbitrary choice of milestone for packages like "RCC beams and slabs, ground to fourth floor" that genuinely do progress continuously.

Its weakness is that the percent is a judgement. See section 5.

1.5 Why work-package level rather than a rolled-up curve

A single project S-curve can tell a manager that something is wrong. It cannot tell them where, and "the project CPI is 0.87" is not an actionable statement.

Carrying the WBS through the whole calculation means the tool can say instead: the ₹51.20 lakh adverse cost variance is **74% attributable to three substructure packages**, all of which are now complete, so that portion of the variance is final and cannot be recovered; while the superstructure packages that are still running are at CPI 0.886 and 0.891 and *can* still move. That is a different conversation, and it is the one worth having.

This is the main functional difference between this tool and the rolled-up S-curve builders we looked at as prior art (section 3).

---

2. Case study data

2.1 What it is

Project	Academic Block (G+4 RCC), institutional campus
BAC	₹14,00,00,000
Baseline	12 monthly periods
Work packages	19 leaves across 6 WBS branches
Data date	End of Period 6
Position at data date	CPI 0.872, SPI 0.863, SPI(t) 0.893, 24.8% complete

The underlying narrative the figures express:

- Hard rock was encountered in excavation (WBS 2.1) from the second week. Pneumatic breaking and controlled blasting were needed for roughly 60% of the excavated volume, against an estimate assuming ordinary soil. The package finished in Period 5 against a baseline of Period 3 and cost ₹61.0 lakh against a budget of ₹42.0 lakh — CPI 0.689.
- The delay cascaded into footings and foundation RCC, both of which overran and finished late.
- Superstructure therefore started a month late and is running an 18-day slab cycle against a planned 12 days.
- Staircase and lift core RCC has not started. Nothing downstream of the frame has begun.

2.2 Why synthetic data

This was a deliberate choice and it has a cost, so it should be defended.

Why not real project data. Actual cost data at work-package level is commercially sensitive. A contractor's real CPI by package reveals margin structure, and obtaining a genuine time-phased baseline with matched actuals would have required a confidentiality agreement that a four-week student project cannot support. Anonymising it properly — scaling values, renaming packages — would have degraded it to something not meaningfully more real than a well-constructed synthetic set.

What synthetic data buys. Complete control over what the case demonstrates. The project is built to exercise every branch of the method: a package that is complete and over budget (variance final, unrecoverable), a package in progress and over budget (still movable), a package not yet started, a

schedule slip that cascades, and a CPI that stabilises early and stays flat — the empirical regularity that most justifies distrusting an optimistic recovery plan.

What it costs. We cannot claim the tool has been tested against the messiness of real data: missing periods, retrospective corrections, packages whose scope changed mid-project, costs booked to the wrong code. Section 6 of the reflection report treats this as the main gap between this work and something a contractor could deploy.

Honesty about the figures. Budgets are order-of-magnitude plausible for an institutional building of this size built to CPWD DSR-style item rates. They are **assumed**. They are not taken from any tender, estimate or invoice, and no claim is made that they are accurate unit rates.

2.3 How the baseline is phased

Each work package has a baseline start, a baseline finish and a curve shape. Linear packages accrue budget evenly. S-shaped packages follow smoothstep, $3t^2 - 2t^3$, which has zero slope at both ends so a package ramps up and tails off rather than starting and stopping abruptly. That is the usual shape of a construction work package, and aggregating nineteen of them staggered across twelve months is what produces the project S-curve — the curve is an emergent property of the phasing, not a shape imposed on the data.

Generated by `data/generate_case_study.py`, which is deterministic and documented. Rerunning it reproduces the CSVs exactly.

---

3. Prior art

3.1 What we looked at

`Tahir-yamin/excel-scurve-builder` (MIT licence, https://github.com/Tahir-yamin/excel-scurve-builder) — generates Excel S-curve and earned value workbooks from progress CSVs.

We read this repository before designing ours and took three things from it. None of its code was copied; what we took were ideas and hard-won operational knowledge, and it is credited here and in the source files where each is applied.

1. Live formulas rather than pasted values. Its central idea is that the generated workbook is a working tool — type a new contract value and the whole book recalculates — not a static report. We adopted this for both the manual benchmark workbook and the tool's Excel export. It matters because a report a quantity surveyor cannot re-run is a report they will retype, and retyping is where errors enter.

2. The verification argument. Its documentation makes a point we had not considered and that reshaped our validation strategy:

> a self-check that re-implements the intended logic in Python only validates your own understanding — it will agree with your bug.

This is the reason `scripts/validate_benchmark.py` drives the benchmark workbook through **LibreOffice** and reads back what a real spreadsheet engine computed, rather than re-implementing the

Excel formulas in Python and comparing. Had we done the latter, a formula we had misunderstood would have been misunderstood identically on both sides and the validation would have passed.

3. Excel chart defaults that silently produce broken charts. Its `docs/excel-chart-gotchas.md` documents several OOXML defaults that render a broken chart while the XML looks correct: date-like category labels promoting the axis to a date axis and collapsing bars to hairlines; `invertIfNegative` defaulting to true and filling negative bars white; tick labels defaulting to the axis crossing, which on all-negative data is the top of the plot; and a formula returning `""` being plotted as zero rather than skipped. All four apply directly to our variance and S-curve charts and all four are handled explicitly in `evm/excel_export.py`, with a test asserting each override is in place.

Where we went further. That tool rolls each project up to a single percentage curve and lists the absence of a WBS breakdown as a known limitation. Work-package-level EV is the main thing our tool adds, along with Earned Schedule and the four-method forecast range.

3.2 Standard references

- PMI, *A Guide to the Project Management Body of Knowledge*, for the standard definitions of PV, EV, AC, the variances, the indices and the four EAC methods.
- W. Lipke, "Schedule is Different", *The Measurable News*, 2003 — the original statement of Earned Schedule and of the defect in classical SPI that it corrects.
- The empirical regularity that CPI stabilises early in a project, often by the 20% complete point, and rarely recovers. This is widely reported in the earned value literature on large defence and infrastructure programmes. We use it as the justification for treating a flat, below-1.00 CPI as strong evidence against an optimistic recovery plan, and we have stated it as a reported finding rather than something this project has independently verified.

---

4. Data source list

Source	Used for	Status
`data/generate_case_study.py`	The entire case study: WBS, budgets, baseline phasing, reported progress and actual cost	**Synthetic.** Authored for this project. Deterministic and reproducible.
CPWD DSR-style item rates (general knowledge of the rate structure)	Sanity-checking the order of magnitude of work package budgets	**Not quoted.** No rate is reproduced. Used only to judge whether ₹2.10 crore for the RCC frame of a G+4 block is plausible.
PMBOK standard definitions	Formulas for PV, EV, AC, CV, SV, CPI, SPI, EAC 1–4, ETC, VAC, TCPI	Reference text
Lipke (2003)	Earned Schedule, SPI(t), SV(t), IEAC(t)	Reference paper

Source	Used for	Status
`Tahir-yamin/excel-scurve-builder` (MIT)	Live-formula workbook approach; validation-by-execution argument; Excel chart defaults	Prior art, credited. No code reused.

No external dataset was downloaded, scraped or incorporated. The project has no data licensing or privacy exposure, which is itself a consequence of the synthetic-data decision in section 2.2.

---

5. The limitation that governs the method

Everything above describes arithmetic. The arithmetic has been validated to nine decimal places against an independent implementation.

None of it addresses the input. **Earned value is percent complete multiplied by budget, and percent complete is a judgement made at site by a person who may have an interest in the answer.** A package reported at 60% when it is really 40% produces an earned value wrong by lakhs, and the tool will report the resulting CPI with six-figure precision and a green tick where it belongs.

No spreadsheet, no dashboard and no amount of validation can detect this. Only a measurement at site can. The practical consequence for anyone deploying such a tool is that the governance around *how percent complete is assessed and by whom* matters more than the software, and is the part most likely to be neglected because it is the part that cannot be bought.

---

6. On the use of AI in this project

The project was built with AI assistance throughout, logged in full in `docs/05-ai-usage-log.md`. One methodological decision deserves stating here because it shaped the tool.

The monthly commentary is generated from templates and thresholds, deterministically. It does not call a language model. This was a deliberate choice, not a limitation we failed to overcome, for three reasons:

1. **Reproducibility.** The same inputs must produce the same sentence every month. "The model phrased it differently this time" is not a defence in front of a client.
2. **It cannot invent.** A language model given a table of indices will readily produce a fluent paragraph containing a number that is not in the table. In the commentary module there is no path by which a figure can reach the text without having been computed first, and a test asserts it.
3. **It is auditable.** Every sentence traces to a threshold written down in `evm/commentary.py` and justified there. A reader who disagrees with the "within 5% is noise" band can find it and argue with it.

Where AI genuinely earned its place was upstream: drafting those templates, checking formula logic, generating code, and catching an arithmetic slip in our own hand calculation. Not in generating the numbers a decision rests on. That distinction is this project's main argument about AI in professional practice and is developed in the reflection report.